Introduction

In this lab, you will write and test code on the ESP32 that modulates an audio signal using pulse width modulation (PWM), allowing the ESP32 to output directly to a speaker. Additionally, you will use the 100 individually addressable WS2812 RGB LEDs to display a visualization of the audio signal.

This lab requires low-level programming of both the **LEDC** and **RMT** peripherals on the ESP32. The **RMT** peripheral generates a signal consisting of bits that are each represented as a time duration spent at logic-1 followed by a time duration spent at logic-0. The **LEDC** peripheral generates a PWM signal that is preconfigured with a specific frequency.

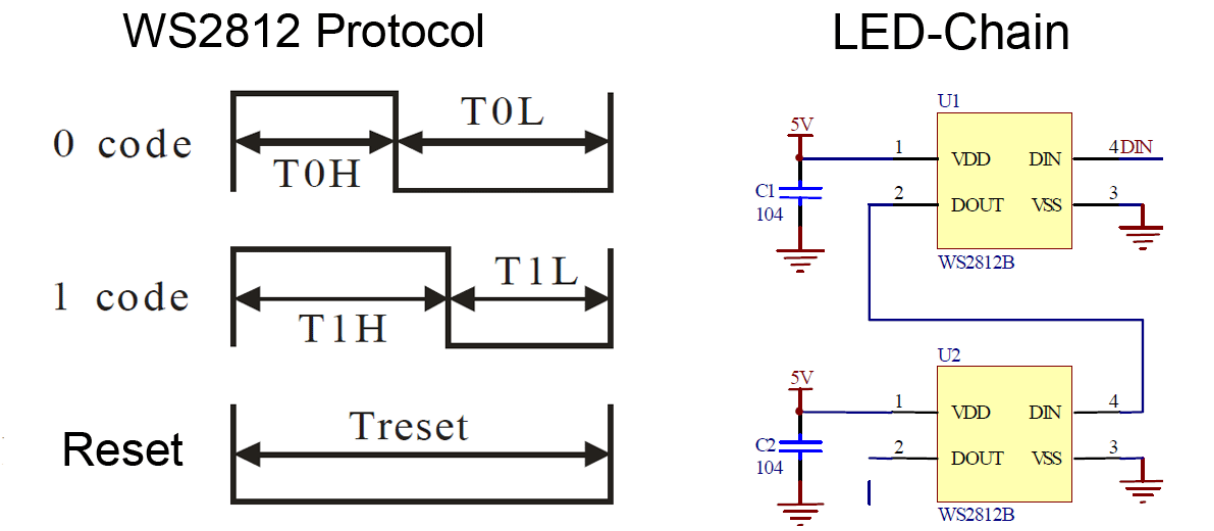
WS2812 Addressing

The WS2812 is a single device that includes red, green, and blue LEDs and an integrated controller that accepts a serial data stream for setting the brightness of the individual LED colors. Upon receiving the first 24-bit RGB value on power-up or after a reset signal is received, the WS2812 configures its LEDs with the corresponding 24-bit color. Afterward, subsequently-received 24-bit values are forwarded on to the next WS2812 in series. This allows a single master to individually control all the WS2812s that are connected in a string.

The WS2812 protocol is asynchronous and consists only of a single “data in” signal called DIN and “data out” signal called DOUT. On power-on, the LEDs are off. At this point the WS2812 is in an “accept” state, in which it monitors its DIN port for a 24-bit value with which to set the three LED intensities (red, green, and blue). Each bit is conveyed to the WS2812 as a high (level-1) period followed by a low (level-0) period. These periods are set depending on if the transmitted bit is a 1 or a 0.

Sending a 1-bit requires a high period, or **T1H**, of 800 ns and a low period, or **T1L** of 450 ns, while a 0-bit requires a high period, or **T0H**, of 400 ns and a low period, or **T0L**, of 850 ns. This is shown in the figure below.

Copyright 2026 Jason D. Bakos and Alex Anderson-McLeod



(Source: https://cpldcpu.files.wordpress.com/2014/01/ws2812_protocol.png)

From the perspective of an individual LED, when in the "accept state", the DOUT port is driven to a constant logic-0. After 24 bits are received, the WS2812 transitions into the "pass-through state", where incoming bits are ignored for the purpose of setting the LED intensities and are instead forwarded to the DOUT port, allowing a downstream WS2812 to use the bits to set its LED intensities.

If no bits arrive on DIN for at least 50,000 ns, the WS2812 transitions back into "accept" state, allowing its LED intensities to be set again. In other words, not driving DIN for 50 microseconds resets the controller but does not turn off the LEDs, which will remain at their previous setting.

This means that, for playing a video using these LEDs, a controller must budget enough time to configure all the LEDs in the frame and have enough time left over to include the 50 us idle period to reset for the next frame. Thus, for a desired reconfiguration rate of, for example 30 times per second (30 fps), there can be no more than (1/30 - 50us) / (850 ns + 400 ns) = 26627 LEDs, forming, for example, a 30 fps 16:9 display having resolution:

```
sqrt(26627 * 16/9) x sqrt(26627 * 9/16) = 217 x 122 pixels
```

Note that this limitation could be overcome by using multiple ESP32 controlling a single display, or potentially from a single ESP32 using more than one RMT channel, though there may be other limiting factors (see below).

ESP32 Remote Control Transceiver (RMT)

The ESP32 RMT (Remote control) peripheral is designed for transmitting/receiving digitally-modulated messages for communicating with IR remote controls. In this lab we will only use its transmitting

ability.

The RMT has eight channels. Each channel can be clocked by the 80MHz APB_CLK, a 40MHz XTAL_CLK, or a 17.5MHz RC_FAST_CLK. Whichever reference clock is selected is subjected to a clock divider. The divided clock increments a counter whose output is used to determine whether the channel output is driven to logic-1 or logic-0. The number of clock cycles corresponding to each logic level are programmed into a memory called the RMT RAM.

Initializing the RMT

Initializing the RMT requires setting up four registers.

Two of these, **SYSTEM_PERIP_CLK_EN0_REG** and **SYSTEM_PERIP_RST_EN0_REG**, are global registers that allow the RMT peripheral to be enabled and taken out of reset.

The other two registers, **RMT_SYS_CONF_REG** and **RMT_CH0_CONF0_REG**, are used to configure RMT-specific settings.

(1) In **SYSTEM_PERIP_CLK_EN0_REG**:

Register 17.4. SYSTEM_PERIP_CLK_EN0_REG (0x0018)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)	SYSTEM_ADC2_ARB_CLK_EN	SYSTEM_SYSTIMER_CLK_EN	SYSTEM_APB_SARADC_CLK_EN	(reserved)			SYSTEM_UART_MEM_CLK_EN	SYSTEM_USB_CLK_EN	(reserved)	SYSTEM_I2S1_CLK_EN	SYSTEM_PWM1_CLK_EN	SYSTEM_CAN_CLK_EN	SYSTEM_I2C_EXT1_CLK_EN	SYSTEM_PWM0_CLK_EN	SYSTEM_SPI3_CLK_EN	SYSTEM_TIMERGROUP1_CLK_EN	(reserved)	SYSTEM_TIMERGROUP_CLK_EN	(reserved)	SYSTEM_LEDC_CLK_EN	SYSTEM_PCNT_CLK_EN	SYSTEM_RMT_CLK_EN	SYSTEM_UHCI0_CLK_EN	SYSTEM_I2C_EXT0_CLK_EN	SYSTEM_SPI2_CLK_EN	SYSTEM_UART1_CLK_EN	(reserved)	SYSTEM_I2S0_CLK_EN	SYSTEM_UART_CLK_EN	SYSTEM_SPI01_CLK_EN	(reserved)	
0	1	1	0	0	0	0	1	1	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	1	1	0	0	1	1	0	Reset

Figure 1: SYSTEM_PERIP_CLK_EN0_REG

Turn on the RMT; set SYSTEM_RMT_CLK_EN bit (bit 9) to 1.

(2) In in **SYSTEM_PERIP_RST_EN0_REG**:

Register 17.6. SYSTEM_PERIP_RST_EN0_REG (0x0020)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)	SYSTEM_ADC2_ARB_RST	SYSTEM_SYSTIMER_RST	SYSTEM_APB_SARADC_RST	(reserved)			SYSTEM_UART_MEM_RST	SYSTEM_USB_RST	(reserved)	SYSTEM_I2S1_RST	SYSTEM_PWM1_RST	SYSTEM_CAN_RST	SYSTEM_I2C_EXT1_RST	SYSTEM_PWM0_RST	SYSTEM_SPI3_RST	SYSTEM_TIMERGROUP1_RST	(reserved)	SYSTEM_TIMERGROUP_RST	(reserved)	SYSTEM_LEDC_RST	SYSTEM_PCNT_RST	SYSTEM_RMT_RST	SYSTEM_UHCI0_RST	SYSTEM_I2C_EXT0_RST	SYSTEM_SPI2_RST	SYSTEM_UART1_RST	(reserved)	SYSTEM_I2S0_RST	SYSTEM_UART_RST	SYSTEM_SPI01_RST	(reserved)	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Figure 2: SYSTEM_PERIP_RST_EN0_REG

Pull RMT out of reset; set SYSTEM_RMT_RST (bit 9) to 0 (may want to set to 1, then to 0 to ensure reset).

(3) In the **RMT_SYS_CONF_REG register**:

Register 37.7. RMT_SYS_CONF_REG (0x00C0)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
RMT_CLK_EN	(reserved)				RMT_SCLK_ACTIVE	RMT_SCLK_SEL		RMT_SCLK_DIV_B						RMT_SCLK_DIV_A						RMT_SCLK_DIV_NUM								RMT_MEM_FORCE_PU	RMT_MEM_FORCE_PD	RMT_MEM_CLK_FORCE_ON	RMT_APB_FIFO_MASK	
0	0	0	0	0	1	0x1		0x0						0x0						0x1								0	0	0	0	Reset

Figure 3: RMT_SYS_CONF_REG

Set RMT_APB_FIFO_MASK to 0.

Set RMT_CLK_EN (bit 31) to 1.

Set RMT_MEM_CLK_FORCE_ON (bit 1) to 1.

Set RMT_SCLK_DIV_NUM to 0 (sets clock divider to 1; NUM+1).

(4) In the **RMT_CH0_CONF0_REG register**:

Register 37.3. RMT_CH*n*CONF0_REG (*n*: 0-3) (0x0020+0x4***n*)

(reserved)	RMT_DMA_ACCESS_EN_CH3 (reserved for *n*:0-2)	RMT_CONF_UPDATE_CH*n*	(reserved)	RMT_CARRIER_OUT_LV_CH*n*	RMT_CARRIER_EN_CH*n*	RMT_CARRIER_EFF_EN_CH*n*	RMT_MEM_SIZE_CH*n*	RMT_DIV_CNT_CH*n*	RMT_TX_STOP_CH*n*	RMT_IDLE_OUT_EN_CH*n*	RMT_IDLE_OUT_LV_CH*n*	RMT_MEM_TX_WRAP_EN_CH*n*	RMT_TX_CONTI_MODE_CH*n*	RMT_APB_MEM_RST_CH*n*	RMT_MEM_RD_RST_CH*n*	RMT_TX_START_CH*n*	
31 … 26	25	24	23	22	21	20	19 … 16	15 … 8	7	6	5	4	3	2	1	0	
0 0 0 0 0 0 0	0	0	0	1	1	1	0x1	0x2	0	0	0	0	0	0	0	0	Reset

Figure 4: RMT_CHn_CONF0_REG_S3

Set RMT_CARRIER_EN_CH0 to 0 to disable RMT carrier frequency.

Set RMT_DIV_CNT_CH0 to 1.

Set RMT_IDLE_OUT_EN_CH0 to 1 to drive the output to logic-0 when the RMT is not transmitting (this enables the WS2812 reset functionality).

Set the RMT_IDLE_OUT_LV_CH0 to 0 to set the output to 0 when the RMT is idle.

Set RMT_CONF_UPDATE_CH0 (bit 24) to 1.

Connecting the RMT and LEDC Peripherals to Output Pins

In its output mode, the GPIO system is configured in the following manner:

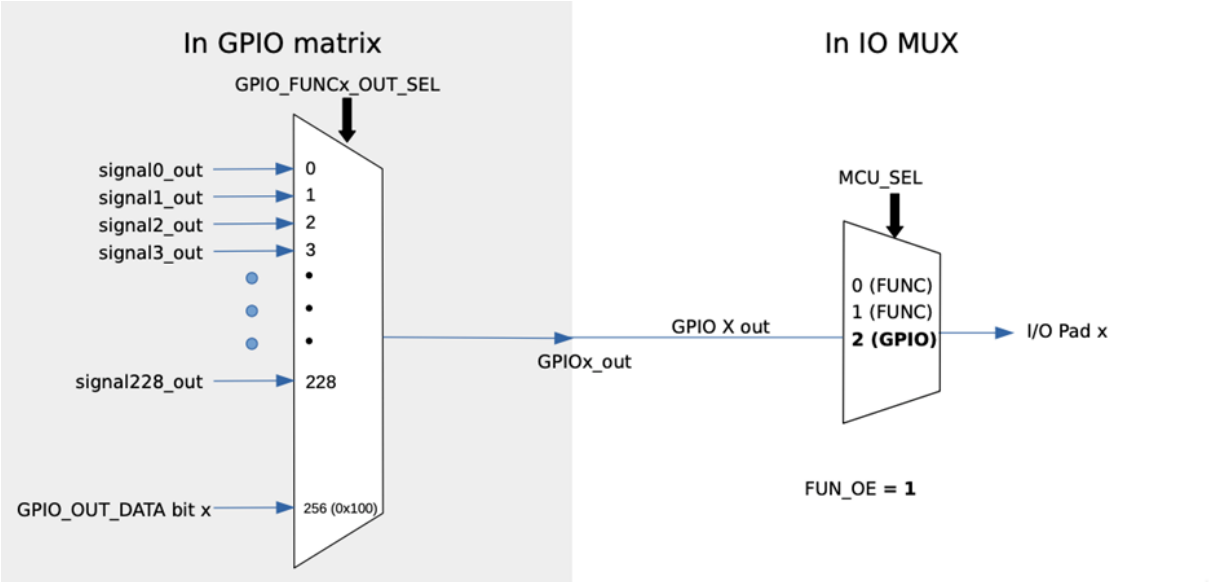


Figure 5: GPIO matrix and IOMUX

The GPIO matrix selects one of the 184 ESP32-S3's internal peripheral signals to route to each of the 45 user-programmable GPIO pins. Thus, there are 45 *GPIO_FUNCn_OUT_SEL_CFG_REG* registers (where n is 0 to 44), each corresponding to a GPIO pin and each containing a field called *GPIO_FUNC_OUT_SEL* for selecting the pin's function.

Setting GPIO_FUNC_OUT_SEL to 256 allows the pin to be controlled via software, allowing for "bit banged" functionality, while setting it to 81 selects the RMT peripheral and setting it to 73 selects the LEDC peripheral.

Likewise, the GPIO_FUNCn_OUT_SEL_CFG_REG register for each of the 45 pins has its own address, shown below for this lab.

pin	register	function for this lab	GPIO_FUNC_OUT_SEL
14	GPIO_FUNC14_OUT_SEL_CFG_REG	LEDC	73
33	GPIO_FUNC33_OUT_SEL_CFG_REG	RMT	81

- GPIO_FUNCx_OEN_INV_SEL controls if the output is inverted (set to 0)
- GPIO_FUNCx_OEN_SEL determines if the output enable is controlled by a separate output enable register or by the peripheral (set to 0)
- GPIO_FUNCx_OUT_SEL determines which peripheral (function) to connect to our pin

pin	register	MCU_SEL
14	IO_MUX_GPIO14_REG	2
33	IO_MUX_GPIO33_REG	2

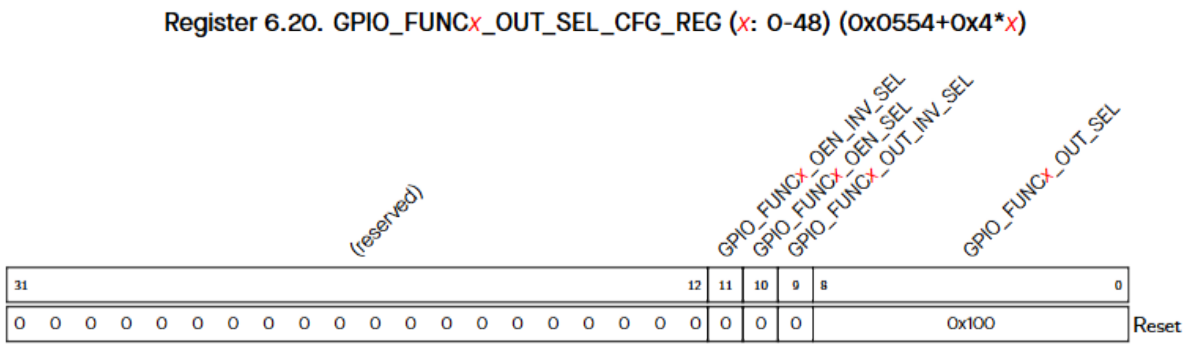


Figure 6: GPIO_FUNCn_OUT_SEL_CFG_REG

The GPIO matrix must pass additionally through each pin's IOMUX before finally hitting the pin. The IOMUX allows for the attachment of pull-up/pull-down resistors and allow higher-speed signals to bypass the GPIO matrix to avoid the GPIO matrix performance overhead. However, you do not need to change any of its control signals.

Programming the RMT Output Data

RMT RAM is divided into eight blocks of 48 words each, one block per channel. Each word holds transmission data in the following format:

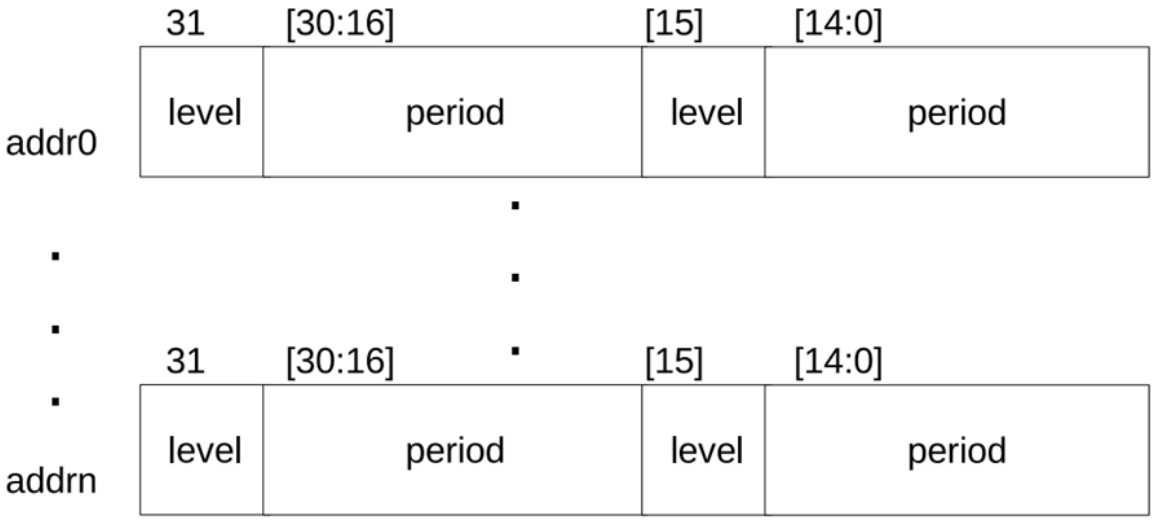


Figure 7: RMT RAM

The level is either 0 or 1, and the period is given in clock cycles. For example, an entry containing the values “0, 5, 1, 10” should cause the output to go low for 5 cycles and then high for 10 cycles. The RMT will stop reading data when it encounters a word where one of the periods is 0.

Steps for Transmitting on RMT

The RMT RAM contains enough entries for 384 pulses, although we recommend using the software to update the RMT RAM after every 24 bits are transmitted (e.g. after configuring each LED). This way, the software will write 24 entries plus a zero termination into the RMT RAM, trigger the RMT to transmit, perform other tasks during the 30 us required for the transmission, and then repeat to setup the next LED. After all LEDs are set up, the software will pause for at least 50 us to cause all LEDs to re-enter the “accept” state.

To perform a transmission:

(1) write 24 entries in the RMT RAM,

(2) write zero termination in the 25th entry in the RMT RAM,

(3) set CONF_UPDATE (bit 24) to 1 in the RMT_CH0_CONF0_REG register,

(4) set RMT_TX_START_CH0 (bit 0) to 1 in the RMT_CH0_CONF0_REG register.

To wait for completion:

(1) read the RMT_INT_RAW_REG register, check if the RMT_CH0_TX_END_INT_RAW bit is set

(2) if yes, set the RMT_TX_END_INT bit in the RMT_INT_CLR_REG register (suggest you also do this when initializing the RMT)

Register 15.3. RMT_INT_RAW_REG (0x00A0)

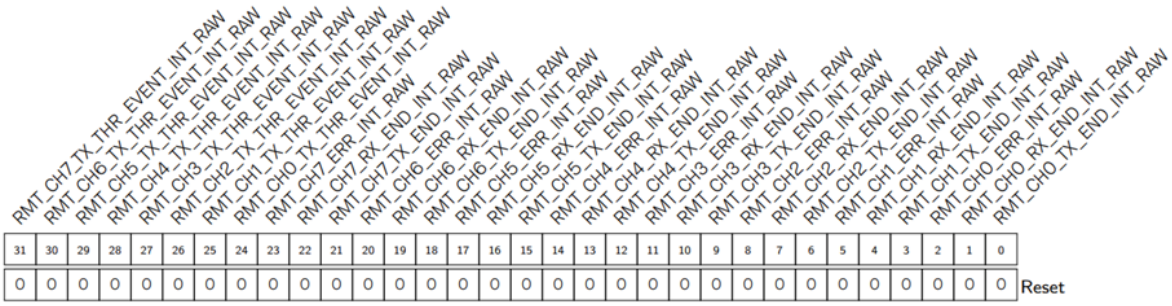


Figure 8: RMT_INT_RAW_REG

Register 15.6. RMT_INT_CLR_REG (0x00AC)

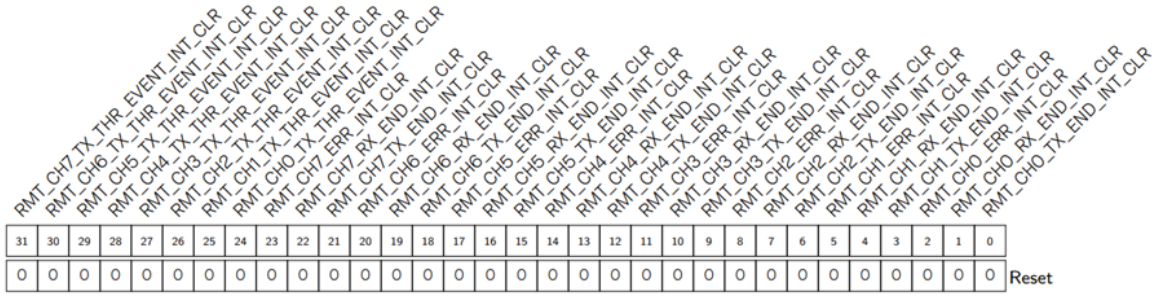


Figure 9: RMT_INT_CLR_REG

Summary of RMT Registers

register	address	purpose
SYSTEM_PERIP_CLK_EN0_REG	0x600C0018	turns on RMT
SYSTEM_PERIP_RST_EN0_REG	0x600C0020	disable reset of RMT
RMT_SYS_CONF_REG	0x600160C0	enable RMT RAM
RMT_CH0_CONF0_REG	0x60016020	set clock divider, disable carrier

register	address	purpose
RMT_INT_RAW_REG	0x60016070	check if RMT is finished transmitting
RMT_INT_CLR_REG	0x6001607C	clear the interrupt indicating completion
RMT_RAM	0x60016800	array of 25 values to define output pattern

Converting an Audio File

We will provide a Python script that converts a .wav file to a C header file containing a constant array called `sampleArray` containing the first 1 million samples. The file also contains a variable called `sampleRate` set to the sample rate.

LEDC

The ESP32 LEDC (LED PWM Control) peripheral generates PWM (pulse-width modulation) signals that can be used for a variety of purposes. As the name implies, it's intended to be used for driving an LED to vary its brightness, but it can be used for anything that needs a PWM signal, including a speaker, motor, or power MOSFET.

As with the RMT, the LEDC can use the 80MHz APB_CLK, a 40MHz XTAL_CLK, or a 17.5MHz RC_FAST_CLK. You can use 80 MHz clock for this lab. This clock is divided by the PWM clock divider, which is a (18, 8) fixed-point value.

Then the PWM period is the time required to flip a counter having N bits, where N is defined by the duty resolution. For example, the PWM period for the 80 MHz clock with a divider of 1 and 8-bit duty resolution would be (8e7/1)/256 = 312.5 KHz.

Initializing LEDC

(1) Turn on LEDC: set bit 11 in SYSTEM_PERIP_CLK_EN0_REG
(2) Take LEDC out of reset: clear bit 11 in SYSTEM_PERIP_RST_EN0_REG (or set and clear to ensure reset)

In **LEDC_CONF_REG**:

LEDC_CONF_REG

Set bit 31 to 1 to enable the clock to the LEDC's registers

Set bit 0 to 1 to select the APB clock.

In **LEDC_TIMER0_CONF_REG**:

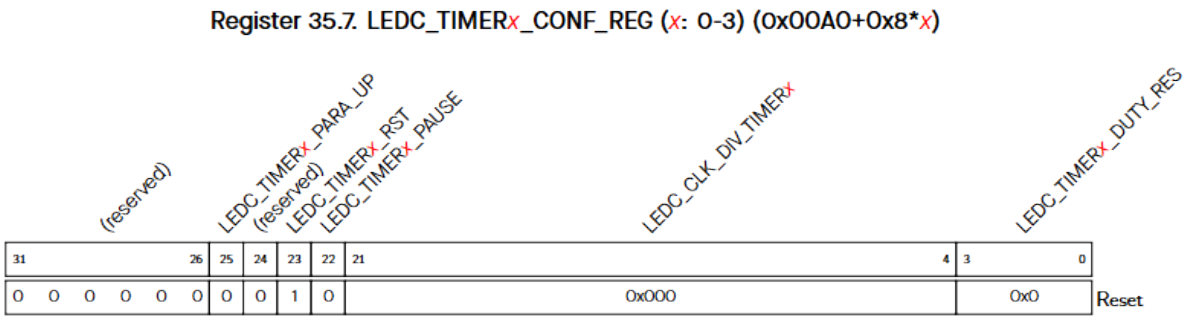


Figure 10: LEDC_TIMERx_CONF_REG

First, compute the clock divider value. Our reference clock is 80 MHz, and we need to divide the clock to make it match our audio sample rate, which we will denote as *R*. The divider is set as a whole part and a fractional part. The whole part is computed as floor(80e6 / R). The fractional part can be computed as the nearest multiple of 1/256, so you can compute it as (80e6/R - floor(80e6/R))*256. Once you have this:

(1) Set LEDC_DUTY_RES to 8 (as given by the sample size in your audio file).

(2) Set LEDC_TIMER0_RST to 0 (bit 23).

(3) Set LEDC_CLK_DIV_TIMERx to (10.8 bit divider value calculated above)

(4) Set LEDC_TIMER0_PARA_UP (bit 25) to commit changes.

In the **LEDC_CH0_CONF0_REG**:

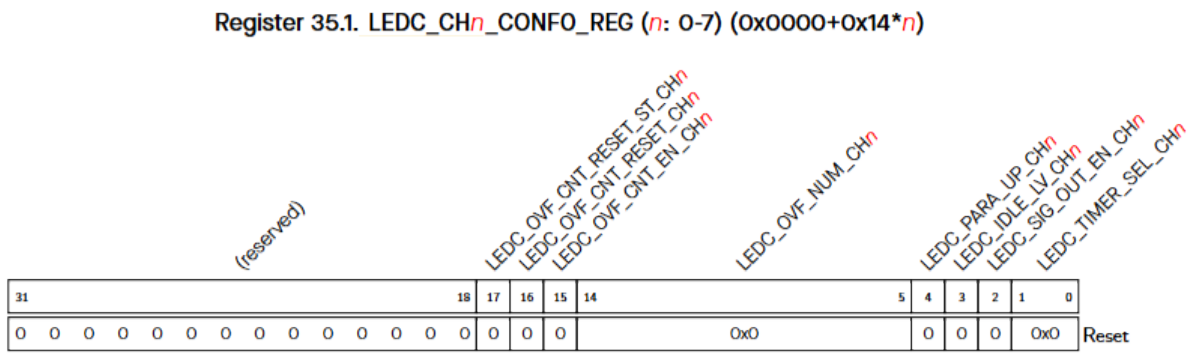


Figure 11: LED_CHn_CONF0_REG

(1) Set bit 2 to enable LEDC.

(2) Set LEDC_TIMER_SEL_CH0 to 0 (selects timer).

(3) Set LEDC_PARA_UP_CH0 (bit 4) to 1.

Sending Data through the LEDC PWM

In **LEDC_INT_RAW_REG** and **LEDC_INT_CLR_REG**:

Register 35.9. LEDC_INT_RAW_REG (0x00C0)

31										20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)											LEDC_OVF_CNT_CH7_INT_RAW	LEDC_OVF_CNT_CH6_INT_RAW	LEDC_OVF_CNT_CH5_INT_RAW	LEDC_OVF_CNT_CH4_INT_RAW	LEDC_OVF_CNT_CH3_INT_RAW	LEDC_OVF_CNT_CH2_INT_RAW	LEDC_OVF_CNT_CH1_INT_RAW	LEDC_OVF_CNT_CH0_INT_RAW	LEDC_DUTY_CHNG_END_CH7_INT_RAW	LEDC_DUTY_CHNG_END_CH6_INT_RAW	LEDC_DUTY_CHNG_END_CH5_INT_RAW	LEDC_DUTY_CHNG_END_CH4_INT_RAW	LEDC_DUTY_CHNG_END_CH3_INT_RAW	LEDC_DUTY_CHNG_END_CH2_INT_RAW	LEDC_DUTY_CHNG_END_CH1_INT_RAW	LEDC_DUTY_CHNG_END_CH0_INT_RAW	LEDC_TIMER3_OVF_INT_RAW	LEDC_TIMER2_OVF_INT_RAW	LEDC_TIMER1_OVF_INT_RAW	LEDC_TIMER0_OVF_INT_RAW	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Figure 12: LEDC_INT_RAW_REG

Register 35.12. LEDC_INT_CLR_REG (0x00CC)

31										20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)											LEDC_OVF_CNT_CH7_INT_CLR	LEDC_OVF_CNT_CH6_INT_CLR	LEDC_OVF_CNT_CH5_INT_CLR	LEDC_OVF_CNT_CH4_INT_CLR	LEDC_OVF_CNT_CH3_INT_CLR	LEDC_OVF_CNT_CH2_INT_CLR	LEDC_OVF_CNT_CH1_INT_CLR	LEDC_OVF_CNT_CH0_INT_CLR	LEDC_DUTY_CHNG_END_CH7_INT_CLR	LEDC_DUTY_CHNG_END_CH6_INT_CLR	LEDC_DUTY_CHNG_END_CH5_INT_CLR	LEDC_DUTY_CHNG_END_CH4_INT_CLR	LEDC_DUTY_CHNG_END_CH3_INT_CLR	LEDC_DUTY_CHNG_END_CH2_INT_CLR	LEDC_DUTY_CHNG_END_CH1_INT_CLR	LEDC_DUTY_CHNG_END_CH0_INT_CLR	LEDC_TIMER3_OVF_INT_CLR	LEDC_TIMER2_OVF_INT_CLR	LEDC_TIMER1_OVF_INT_CLR	LEDC_TIMER0_OVF_INT_CLR	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Figure 13: LEDC_INT_CLR_REG

The LEDC_TIMER0_OVF_INT_RAW bit in the LEDC_INT_RAW_REG indicates that the counter associated with the LEDC channel overflowed and that one period of the PWM signal just completed.

When this bit is set, clear it by writing the LEDC_TIMER0_OVF_INT_CLR.

In the **LEDC_CH0_DUTY_REG**:

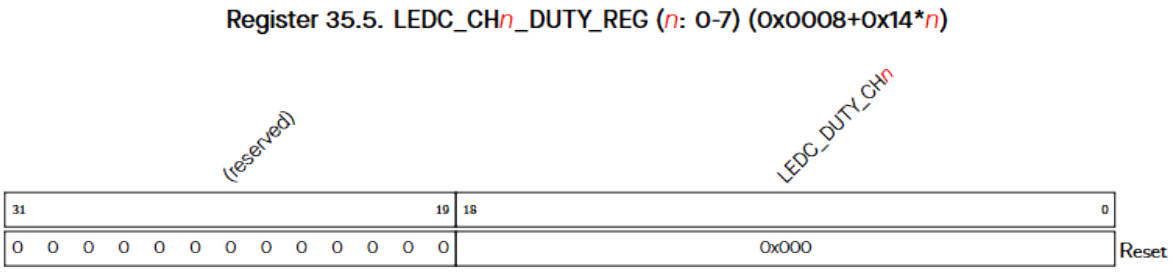


Figure 14: LEDC_CHn_DUTY_REG

Write the duty cycle into the **whole bits** of the LEDC_CHn_DUTY_REG. Be sure to shift the whole bits by 4 bits to the left to set its fraction bits to 0.

In the LEDC_CH0_CONF0_REG (shown above), set the LEDC_PARA_UP_CH0 bit.

In **LEDC_CH0_CONF1_REG**:

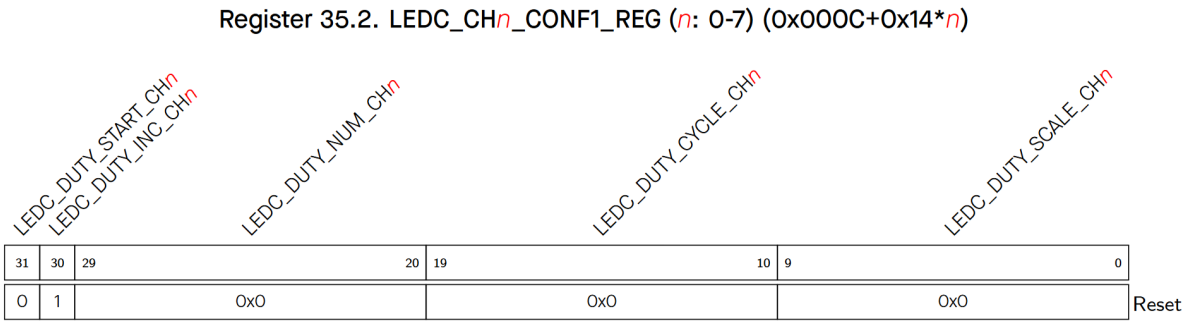


Figure 15: LEDC_CHn_CONF1_REG

In the LEDC_CH0_CONF1_REG, set the LEDC_DUTY_START_CH0 bit.

Summary of LEDC registers

register	address	purpose
LEDC_TIMER0_CONF_REG	0x600190A0	set reference clock, divider, and duty cycle width

register	address	purpose
LEDC_CONF_REG	0x600190D0	Enables and selects LEDC clock
LEDC_CH0_CONF0_REG	0x60019000	idle value, enable, select timer
LEDC_CH0_CONF1_REG	0x6001900C	commits configuration changes
LEDC_CH0_DUTY_REG	0x60019008	duty cycle
LEDC_INT_RAW_REG	0x600190C0	interrupt status
LEDC_INT_CLR_REG	0x600190CC	clear interrupt

Writing Registers

To write to registers, use `REG_WRITE(REG_ADDRESS, DATA);`

To read from registers, use `REG_READ(REG_ADDRESS);`

To set a single bit in a register, do `REG_SET_BIT(REG_ADDRESS, BITMASK);`

To clear a single bit, do `REG_CLR_BIT(REG_ADDRESS, BITMASK);`

Audio Visualization

There are 100 LEDs on the board, connected in row-major order (i.e. the upper-left corner is LED 0, the lower-right corner is LED 99). For this lab, we will visualize the audio on the LEDs by lighting up the LEDs in order from 0 to 99 according to the audio magnitude and also setting their color to indicate the magnitude.

Audio visualization is generally performed using a log-scale plot, .i.e. the magnitude is shown as log10(duty cycle/256), which for duty cycles 1 to 255 gives a range of -5.5 to 0. Each of the 100 LEDs can represent a range of 1.1 starting at -5.5, i.e.

Thus, the number of LEDs lit could be computed as

```
log_val = log10(duty_cycle + 1) / math.log10(256)
num_LEDs_to_light = round(log_val * 100)
```

To make the visualization more dynamic, set the colors of all the LEDs according to the amplitude. You can use your own approach for this or use the colormap below:

```python
# Segment the range into 5 parts (~51 values each)
    if value <= 51:           # Blue to Cyan
        r = 0
        g = value * 5
        b = 255
    elif value <= 102:        # Cyan to Green
        r = 0
        g = 255
        b = 255 - (value - 51) * 5
    elif value <= 153:        # Green to Yellow
        r = (value - 102) * 5
        g = 255
        b = 0
    elif value <= 204:        # Yellow to Orange/Red
        r = 255
        g = 255 - (value - 153) * 5
        b = 0
    else:                     # Orange/Red to Bright Red (or White)
        r = 255
        g = 0
        b = 0 if value < 230 else (value - 204) * 4
```

Requirements

This is a complex lab, so to help in grading and assisting lab groups, we will require that your code follow a few requirements:

(1) All register addresses must be named using `#define`, e.g.

```
`#define    LEDC_TIMER0_CONF_REG        0x600190A0`
```

(2) Write a function `void setup_RMT()` that initializes the RMT.

(3) Write a function `void setup_LEDC()` that initializes the LEDC.

(4) Write a function `void update_PWM (uint32_t initial,uint32_t sample)` that updates the PWM output. The parameter `initial` is a flag that specifies if this is the first time the PWM is being output, and `sample` is the sample (duty cycle value) to output. The function should check the `LEDC_INT_RAW_REG` to determine if it is time to update the PWM value. If not, the function should return without writing to any peripherals unless the `initial` flag is set.

(5) Write a function named `void transmit_led_signal(uint32_t *colors)` that will update all five LEDs to colors as given by the array `colors`.

Important Note

You must supply 12 VDC power to your board to the VBATT input for the audio circuit (the LEDs are powered from USB-C).

Please set your 12 VDC power supply current limit to 100 mA to avoid damaging the speaker.

Submitting Your Code

Submit your code using Blackboard.

Rubric

- 10 points: Meet code requirements
- 10 points: Demonstrate ability to light LEDs
- 10 points: Demonstrate ability to send PWM signals using LEDC
- 10 points: Audio demonstration
- 10 points: Visualization demonstration
- 50 points: Overall demonstration and explanation of code

Copyright 2026 Jason D. Bakos and Alex Anderson-McLeod